Fine-tuning intro for practitioners and the curious

Nikalina Ogorodova · Hyperskill · Oct 2026

Introduction

An example of fine-tuning that worked

A small open model (0.8B parameters), fine-tuned for one job: picking the right option from a list.

Before training	After 2 hours of training	Commercial model built for this
45%	**79%**	83%

Fine-tuning works well for changing how a model answers, not what it knows. This talk is about which problem you have, and the best fix for each.

```
Input:   "Can I move my meeting with the dentist to Friday?"
Options: book · reschedule · cancel · ask a question

Output:  reschedule 0.94 · book 0.03 · cancel 0.02 · ask a question 0.01
```

What is expected from fine-tuning

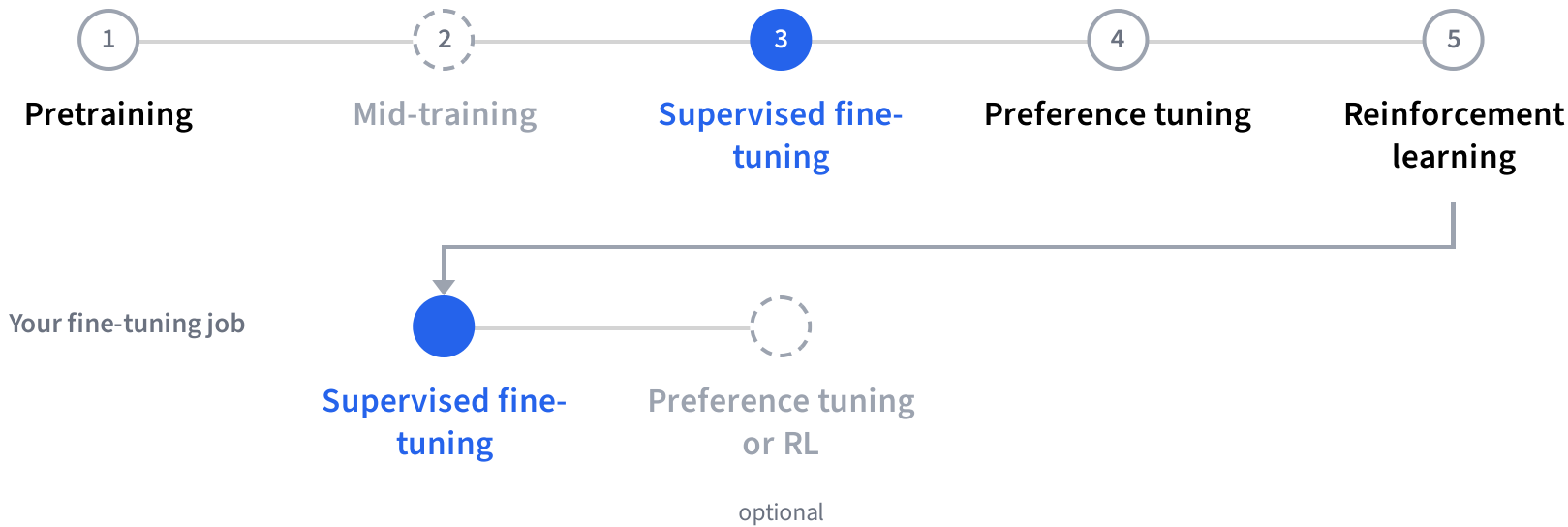
Expectation	What actually happens
"It will learn our docs, products and policies."	Fine-tuning can’t replace retrieval or adding appropriate data sources.
"It will follow our desired output format and rules consistently."	Usually true. One of the main reasons to fine-tune.
“A small fine-tuned model can replace the frontier model in the pipeline."	On a *well-defined repetitive task with clear right answers*, yes.
“Our data stays private because it’s our model.”	Self-hosting keeps data away from outside APIs (but the PII has to be removed)

Today

1. How models are trained, and which stage "fine-tuning" refers to
2. What to try before fine-tuning (a general step sequence)
3. When it's suitable to add a fine-tuned model and what it needs to work as expected

Part 1: How models are trained, through their data

The training pipeline



A typical fine-tuning job adds one more round of supervised training on top of a model that has already been through the 1-5 steps from the first line.

Each stage learns from different data

Stage	What the data looks like	The source
Pretraining	Raw text	Mostly the internet, collected and filtered by the lab
Mid-training	Curated higher-quality text	Selected by the lab
Supervised fine-tuning	Example conversations: a prompt and the ideal response	People, or a stronger model
Preference tuning	One prompt, two responses, the better one marked	People, or an AI judge, choosing between responses
Reinforcement learning	A task plus a way to score attempts	The model writes its own attempts; code (tests, an answer key) or a judge model scores them

Pretraining: predict the next word, trillions of times (Stage 1)

```
The Eiffel Tower is a wrought-iron lattice tower in Paris, France.
It was completed in 1889 for the World's Fair...

Refund policies vary between companies. Most software subscriptions
allow cancellation at any time, but refunds are typically...

def calculate_total(items):
    return sum(item.price for item in items)
```

- Just documents, no special format: questions and answers appear only the way they do on web pages
- Objective: **predict the next token** (roughly a word piece)
- Scale: Llama 3 trained on over 15 trillion tokens of filtered text

Meta, The Llama 3 Herd of Models, 2024 ↗

What the model has after pretraining (Stage 1)

- **Knowledge:** most of what it will ever know is learned here
- Stored as **fuzzy recall**: facts seen thousands of times are recalled well; rare facts poorly; facts it never saw, not at all
- **Not yet behavior:** it continues text; it doesn't follow instructions or answer reliably

Data in	The model learns	Can you run this step?
Raw text	Language and world knowledge	Not at a useful scale: you start from a released model

Talking to a base model (Stage 1)

```
Prompt:  The capital of France is

Output:  Paris, which is also its largest city...
```

```
Prompt:  What is the capital of France?

Output:  (varies) Sometimes "Paris." Sometimes more quiz questions,
         or a forum reply. Whatever usually follows text like this.
```

The knowledge is injected at this stage. Answering reliably and in the expected manner is a separate behavior, taught in the next stages.

Labs usually release two versions: the **base model** (shown here, after pretraining only) and the **instruct model** (after all the later stages). Most people use and fine-tune the instruct one.

Qwen3.5-4B Base (730K downloads in the last month)↗
Qwen3.5-4B (7.49M downloads in the last month)↗

Mid-training: the lab’s final pass (Stage 2)

Still next-token training, but on a smaller, carefully chosen mix:

- **Higher-quality text:** textbooks, math, code, some question-and-answer examples
- **Long documents,** to extend how much text the model can handle at once
- **Increasingly synthetic:** a growing share is written by stronger models

Data in	The model learns	Can you run this step?
Curated text	Sharper reasoning, code and math; longer context	No: it’s part of building the base model. Your version is continued pretraining

Continued pretraining: more raw text, from your domain (Stage 1 + 2)

What it is: more next-token training on a base model, which has already been through pretraining and mid-training, using raw text from your domain.

Good fit: a field with its own way of writing

```
Pt is a 67yo M w/ hx of COPD, T2DM.
Presented w/ 3d productive cough,
febrile 38.6°C, SpO2 91% on RA.
CXR: RLL consolidation. Started
empirical abx per local protocol;
reassess in 48h.
```

Poor fit: a few company documents

```
Acme Cloud Refund Policy, v4.2
Customers on monthly plans may
request a full refund within 30
days of their first payment...
```

- Same training as pretraining: **predict the next token**, now on your own raw text
- Usually mixed with general text so the model keeps what it knew
- Needs a lot of text: hundreds of millions of tokens, not a folder of documents

SFT at a lab

```
User:      What's the capital of France?
Assistant: The capital of France is Paris.

User:      Can you help me write a polite
           email declining a meeting?
Assistant: Of course! Here's a draft: ...
```

SFT in your fine-tuning job

```
{"messages": [
  {"role": "system",
   "content": "You are Acme's support assistant.
              Policy: monthly plans can be refunded
              within 30 days of first payment."},
  {"role": "user",
   "content": "Can I get a refund?
              I signed up 2 weeks ago."},
  {"role": "assistant",
   "content": "Yes! You signed up 2 weeks ago, so
              you're within the 30-day window..."}
]}
{"messages": [ ... ]}
{"messages": [ ... hundreds more ... ]}
```

One example per line in the real file; wrapped here for readability.

What supervised fine-tuning teaches (Stage 3)

- **How** to answer: format, tone, length, when to refuse
- Not a reliable way to add facts: the model already knew Paris; SFT taught it to answer
- If an example states something the model doesn’t know, it learns to state things it doesn’t know, confidently

Data in	The model learns	Can you run this step?
Example conversations	Behavior: format, tone, consistency	**Yes: this is usually what "fine-tuning" means**

Preference tuning: which answer is better? (Stage 4)

```
User: Explain what a mortgage is.

Answer A: A mortgage is a loan used to buy property, where the
          property itself serves as security. You repay it monthly...   ✓ preferred
Answer B: Mortgages are financial instruments subject to amortization
          schedules and lien provisions under...
```

Data in	The model learns	Can you run this step?
Answer pairs, better one marked (by people or an AI judge)	Which answers are better: clearer, more helpful, safer	Yes (e.g. DPO); useful when judging answers is easier than writing them

Reinforcement learning: learn by trial and error (Stage 5)

```
Task:   A train leaves at 9:40 and arrives at 13:15. How long is the trip?
Check:  final answer must equal "3 hours 35 minutes"

Task:   Write a function that sorts a list of dates.
Check:  run these 12 tests; all must pass
```

- Nobody writes example solutions: the model makes attempts, and the ones that score well are reinforced
- Reasoning models (OpenAI o1, DeepSeek-R1) learned step-by-step reasoning largely this way

Data in	The model learns	Can you run this step?
Tasks + a way to score attempts	Skills: math, code, multi-step reasoning	Yes; easiest when a program can check the answers

Not a new idea

That fine-tuning shapes behavior, not knowledge, has been the working view since 2023. LIMA called it the "Superficial Alignment Hypothesis."

- **LIMA:** 1,000 curated examples were enough to make a capable assistant, so the knowledge can’t come from the examples
- **URIAL:** base and tuned models rank the same top word at most positions; they differ mainly on style words and disclaimers
- **Gekhman et al.:** examples with new facts are learned slowly, and once learned, increase hallucination
- **Ovadia et al.:** retrieval beat training on the same documents

LIMA, Zhou et al., 2023 ↗ · URIAL, Lin et al., ICLR 2024 ↗ · Gekhman et al., EMNLP 2024 ↗ · Ovadia et al., EMNLP 2024 ↗

Part 2: What to try first

Step 0: collect test cases before changing anything

- **Start with 20–50 real inputs**.
- Include the cases that worry you: edge cases, past complaints, known failures
- **Run each one a few times.**
- Keep them separate from any examples you put in the prompt
- **Grow to 100–200+** before comparing options, and add every new failure you find

The fixes, cheapest first

1. **Test a stronger model**
   minutes
2. **A better prompt**
   minutes to an hour
3. **The right information**
   minutes to weeks
4. **Split the task**
   hours to days
5. **Fine-tuning**
   weeks, plus upkeep

Ordered by cost, but start at the fix that matches the failure.

1. Test a stronger model first

Why: find out whether the task can be done at all, before optimizing anything

How: run your test cases through 2–3 models.

Compare: how many cases each gets right, and what each costs per case

- **The strongest works:** you've found the ceiling.
- **Even the strongest fails:** the problem is the prompt, the information, or the task itself.

2. A better prompt

Fixes: the model can do the task but doesn’t know what you want.

Format problems: use the built-in structured output mode. It guarantees valid JSON, though not correct content.

3. Access to the right information

Fixes: the model doesn’t have the facts: your products, policies, customers, anything recent or internal

Signs: wrong or outdated facts, confident inventions about your company, "I don’t have access to that"

Find the gaps: tell the model, *"If you don’t have enough information to answer reliably, say what’s missing."* What it flags is a clue to what to add.

How, from simplest:

- Paste it into the prompt; models now accept long documents
- Too much to paste? Retrieve the relevant parts automatically (retrieval, RAG)
- Live data? Let the model call a tool or API

4. Split the task

Fixes: too much happening in one step: long reasoning, many decisions at once

Signs: even the strongest model, with a clear prompt and all the facts, gets it right only sometimes

How: break the job into steps: extract → check → write. Each one is simpler, and can be tested on its own

5. Fine-tuning

You’ve shown the task can be done, and you have examples of good outputs. Now you need it:

- **cheaper or faster** at high volume
- **smaller**, to run on your own servers or a device
- **more consistent** than prompting can make it

Not "the model can't do it." The question is: "we know what good looks like; now make it efficient and reliable."

The one-slide diagnosis

When the model	The problem is	Try
fails, and you don’t know if the task is doable	unknown ceiling	test a stronger model
does it right sometimes, or misreads the task	instructions	a clearer prompt
gets the content right but the style or format wrong	showing, not telling	examples in the prompt
gets facts wrong or makes them up	missing information	the prompt, retrieval, tools
fails even with all of the above	task too complex	split it into steps

Acme’s support assistant

What went wrong	Fix
Is this even doable?	**Stronger model:** tested 3 models; the strongest handles most cases
Answers run three paragraphs long	**Prompt:** "Answer in two or three sentences."
Tone sounds robotic, despite adjectives	**Examples:** three real replies from the best support agent
States the wrong refund window	**Information:** put the refund policy in the prompt
Mishandles refunds involving plan changes and discounts	**Split:** look up the account → check the policy → write the reply

Part 3: Is fine-tuning worth it?

Fine-tuning makes sense for a narrow, stable task where you know what good output looks like, and you need it cheaper, faster, more consistent, or running where APIs can't.

Context: decision models

Jev (TypeSafe, 2026): a model that **doesn't write text**. You give it a situation and a question with fixed options; it returns a probability for each option.

```
Situation:  "Hi, I was charged twice this month and need this fixed today."

Question:   Which team should handle this?
Options:    billing · technical support · sales · account security

Answer:     billing 0.91 · account security 0.06 · technical support 0.02 · sales 0.01
```

- Built for the small decisions inside software: routing a request, flagging harmful content, checking an answer against a source, scoring against a rubric
- "System One" is from psychology: fast, intuitive judgment, as opposed to slow step-by-step reasoning
- Fast and cheap because it never generates a reply, only scores the options

Deußer et al., Evaluating and Benchmarking the System One Model Jev, 2026 ↗

A great fit: small decision models

Jeff: open 0.8B–2B models that pick from a list of options, fine-tuned from Qwen3.5 and Gemma 4

	Untrained 0.8B	Fine-tuned 0.8B	Jev (commercial)
Accuracy, 5 benchmarks	45.3%	79.1%	83.0%
Time per decision		~22 ms on one GPU	API call

- Trained in about 2 hours on one workstation GPU
- A further fine-tune on ~11k app-specific examples: 31.7% → 95.8%, in under half an hour

firelex/jeff, Sept 2026 ↗

More good fits

Example	Task	Why fine-tuning fits
NuExtract3 (NuMind)	4B model: documents → JSON or Markdown	Fixed output format; huge volume; runs on your own servers
ART email agent (OpenPipe)	14B model searching email, trained with RL on one GPU for under $80	A program can check if it found the right email; beat OpenAI's top model on that task
Jeff adapters	Spam detection: 72% → 98%; contract clause types: 66% → 86%	Narrow labels; plenty of labelled examples

The pattern: a narrow task, a fixed or checkable output, and enough volume to justify owning the model.
NuMind, NuExtract3, 2026 ↗ · OpenPipe ART ↗ · firelex/jeff ↗

When fine-tuning is the right call

1. **A small model doing one job as well as a big one.** The big model already does it; the small one learns to match it, at a fraction of the cost and latency
2. **Consistent output at very high volume.** Strict formats or fixed labels, where even a 2% failure rate costs real money across millions of calls
3. **Behavior that's hard to describe but easy to show.** A house style, with hundreds of good examples, more than can fit in a prompt
4. **Running where APIs can't.** On your own servers, on a device, offline, or under data rules that rule out outside APIs
5. **Skills with an answer the code can check.** Advanced: RLVR

What all five share: they're about how the model behaves, and you know what good output looks like.

What it needs: data

- **Hundreds to thousands of example conversations,** in the format from the supervised fine-tuning slide in Part 1
- **Quality beats quantity:** the model copies mistakes as faithfully as good answers
- **Cover the edge cases** you care about, not just the easy ones
- **Where it comes from:** historical data, like tickets labeled by agents, or a big model's answers, spot-checked by people
- **Check that you're allowed to train on it:** remove personal data; check your provider's terms before training on its model's outputs
- **Plus a held-out test set** the model never trains on: the Part 2 test cases, expanded

What it costs to build and keep alive

Build: preparing and checking data is most of the work. The training run can be cheap: a provider's fine-tuning API, or a small add-on (LoRA) trained on one GPU with tools like Unsloth, in minutes to hours

Run: an open model has to be hosted somewhere; a provider's fine-tuned model may cost more per call than its base model

Maintain: the cost people forget

- The task changes (new categories, a new policy): new data, a new training run. A prompt change takes minutes
- A better base model comes out: your fine-tune stays on the old one. An adapter doesn't carry over; your dataset does
- Old base models get retired, and fine-tunes built on them have to be rebuilt
- Someone has to own the data, the evaluation and the retraining

How to check that it actually helped

Decide what counts as success before training, e.g. "within 3 points of the big model, at a fifth of the cost"

Run the held-out test set (100–200+ cases) on:

Setup	Why
1. Best prompt on a big model	The baseline from Part 2
2. Small model, same prompt, no fine-tuning	Did training add anything?
3. Fine-tuned small model	The candidate
4. For classification: a simple classifier on text embeddings	Do you need an LLM at all?

- **Worth it if** (3) closes most of the gap to (1), at a cost or speed that matters, and clearly beats (2) and (4)
- **Check for regressions:** worse outside the training examples, weaker safety and refusal behavior

Good fit: ticket triage

About 50,000 tickets a day, each turned into JSON: category (from ~40), urgency, product, sentiment, one-line summary

- **Behavior, not knowledge:** all the facts are in the ticket
- **Strong baseline:** a big model already does it, but cost, latency and customer data add up
- **Data exists:** agent-labeled history, or big-model labels checked by people
- **Measurable:** accuracy per field, valid-JSON rate
- **Stable:** categories change a few times a year

Poor fit: "our documentation"

"We fine-tuned on our 300-page product documentation."

- The model picks up the documentation's tone
- It invents details
- Every documentation update means retraining

The fix: retrieval, optionally with a small fine-tune for answer style

The test that separates them

Are the facts in the input, or would they have to live in the model?

Decision checklist

Is fine-tuning the right fix?

1. Have we tried a clear prompt, examples and the right information on a strong model?
2. Is the problem how the model behaves, not what it knows?
3. Do we know what good output looks like, with cost, speed, consistency or deployment as the real issue?
4. Is the task narrow and stable?

Are we ready?

5. Do we have (or can we make) hundreds to thousands of good examples that we're allowed to train on?
6. Do we have a test set, and a success threshold decided in advance?
7. Is someone going to own retraining and monitoring?

Any "no" in the first group: stay on the ladder. A "no" in the second: fix that first, then run a pilot.

Part 4: Wrapping up

Three things to take away

1. **Fine-tuning changes how a model answers, not what it knows.** Facts go in the prompt, retrieval or tools
2. **Diagnose before you fix.** Collect test cases, sort the failures, and try the cheapest fix that matches
3. **Fine-tune when you know what good output looks like,** and you need it cheaper, faster, more consistent or running locally. Plan to maintain it

If you do fine-tune: Unsloth

An open-source tool for fine-tuning open models on a single GPU, including a free cloud notebook (Kaggle/ Google Colab). Unsloth docs ↗

Option	What it is	Best for
Desktop	Native app, no code	Trying it for the first time
Studio	Web interface: pick a model, a method and a dataset; watch training live	A team sharing one GPU machine
Core	Python library	Repeatable training scripts kept in Git

- Supports LoRA, QLoRA and full fine-tuning, plus reinforcement learning (GRPO)
- Exports models to run locally, e.g. in llama.cpp or Ollama

What the work looks like

1. **Pick a base model:** usually an instruct model, small enough for your hardware
2. **Prepare the dataset:** example conversations, one per line, as in Part 1
3. **Train a LoRA:** start from the defaults; minutes to hours on one GPU
4. **Evaluate:** your held-out test set, against the baselines from Part 3
5. **Export and serve:** locally, or on a hosting provider

Unsloth (and similar automated training solutions) make step 3 easy. Steps 2 and 4 are still most of the work.

Thank you

Questions?

Nikalina Ogorodova · Hyperskill · Oct 2026